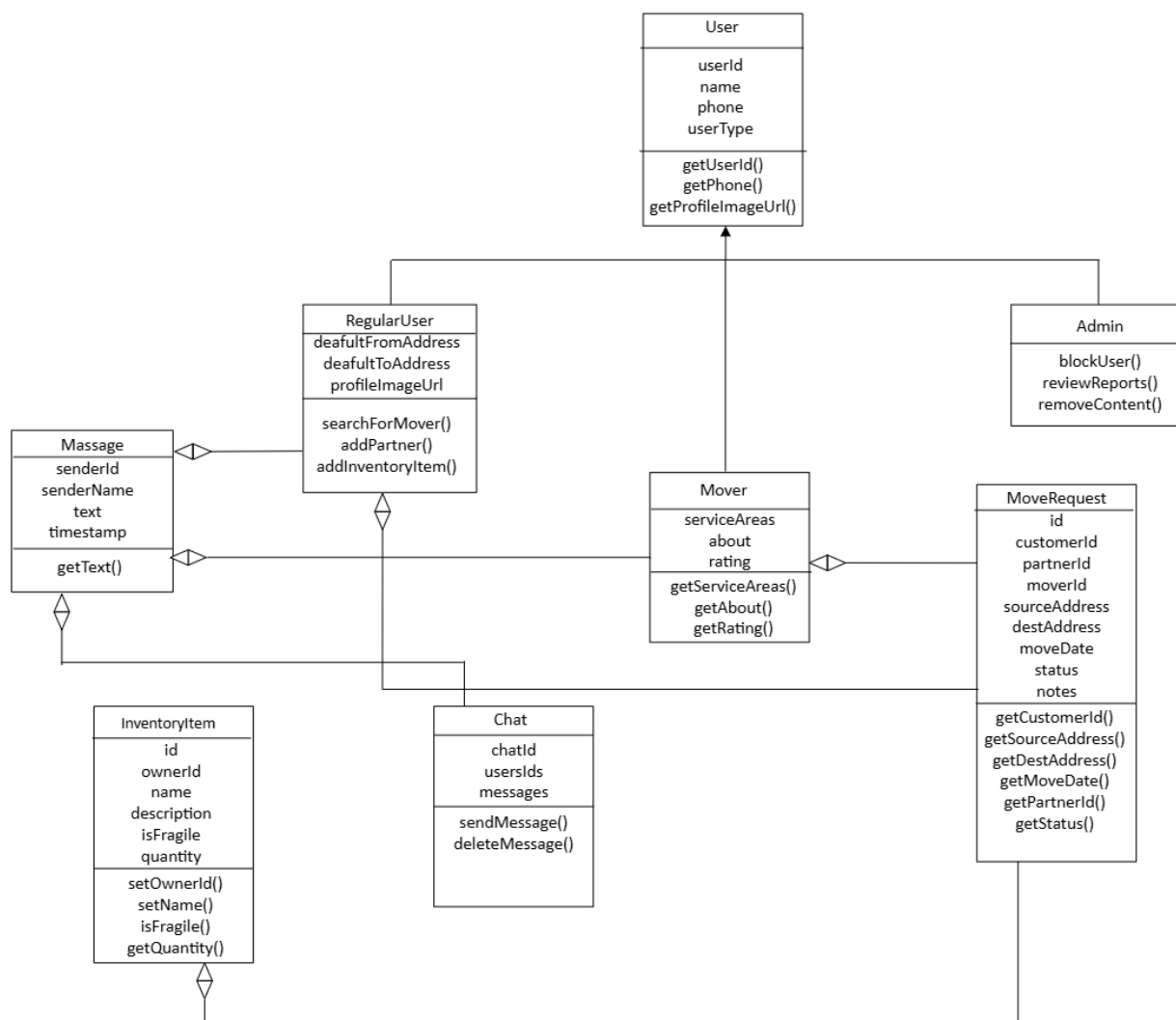


מטלה 3

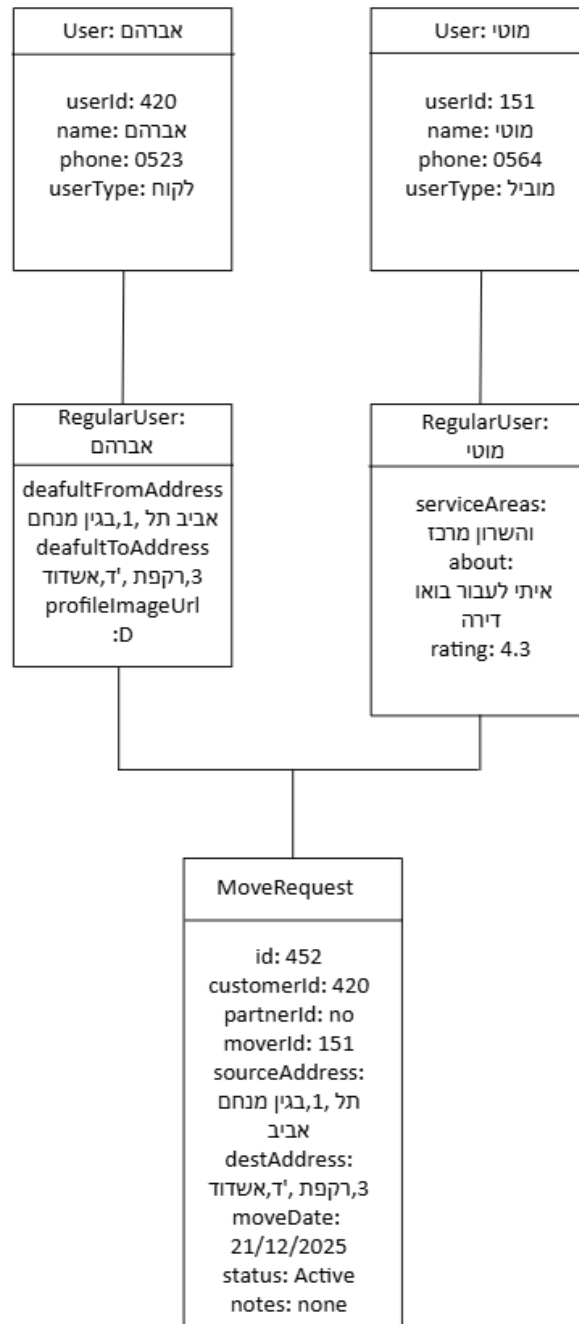
מגישים: עמית ביטון, דוד קיטינברג, שירה בן ארצי, אלדן אבו חגלה

ת"ז: 32223158, 211520127, 322315300, 213763519

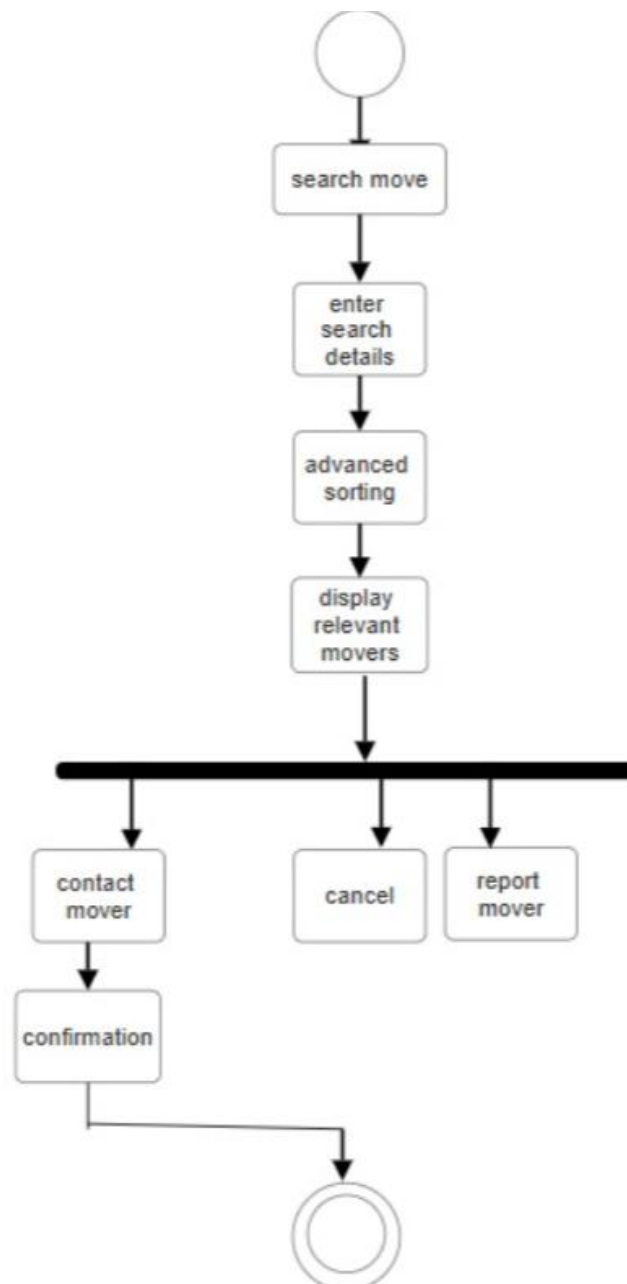
1. Class diagram



2. Object diagram

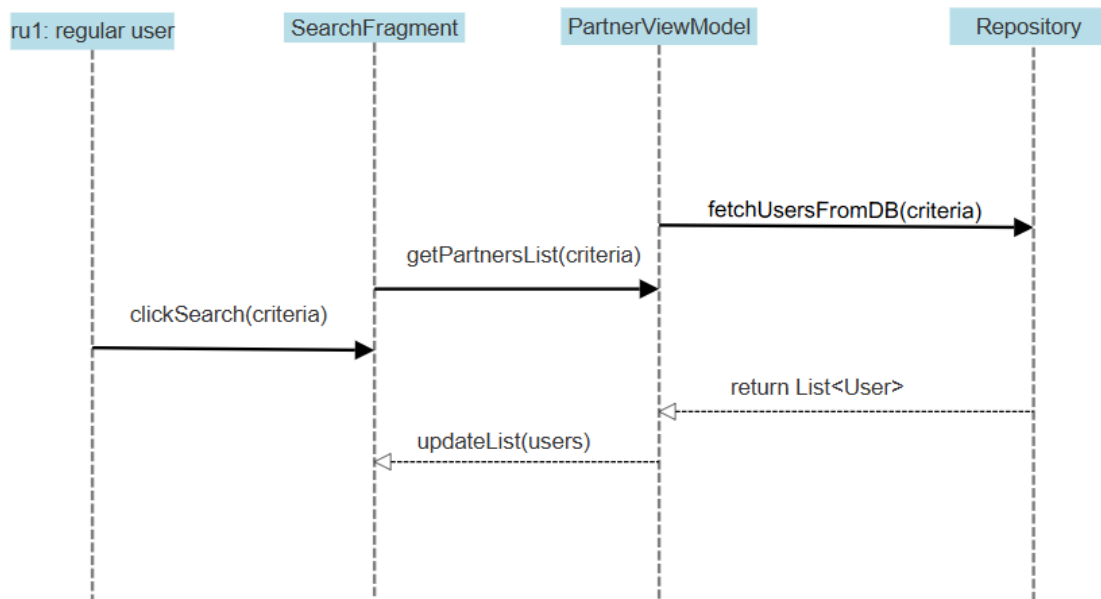


3. Activity diagram

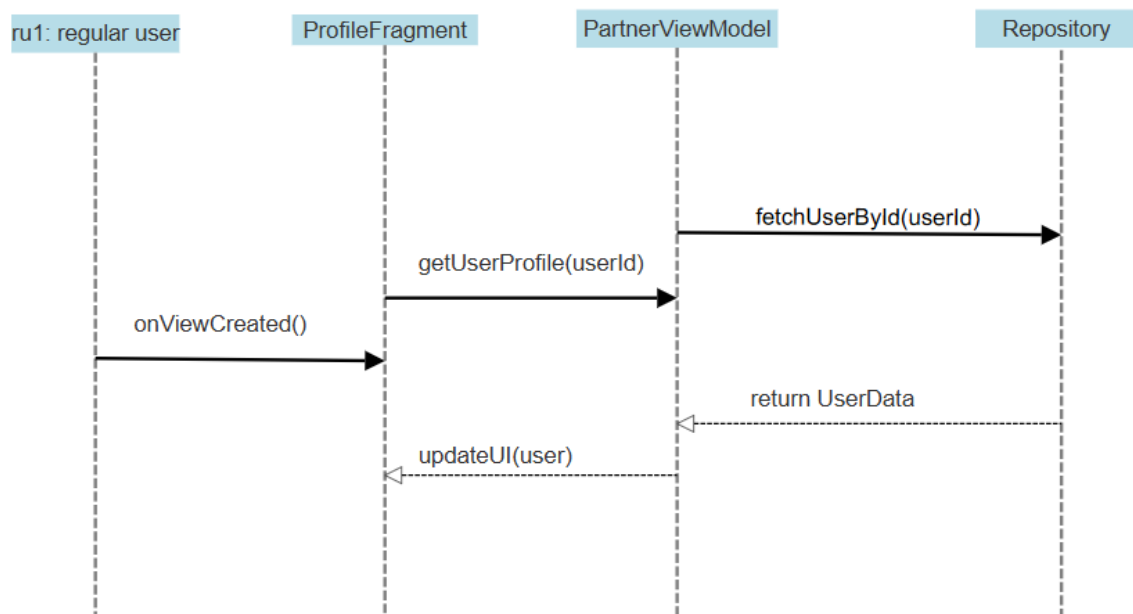


4. Sequence diagram

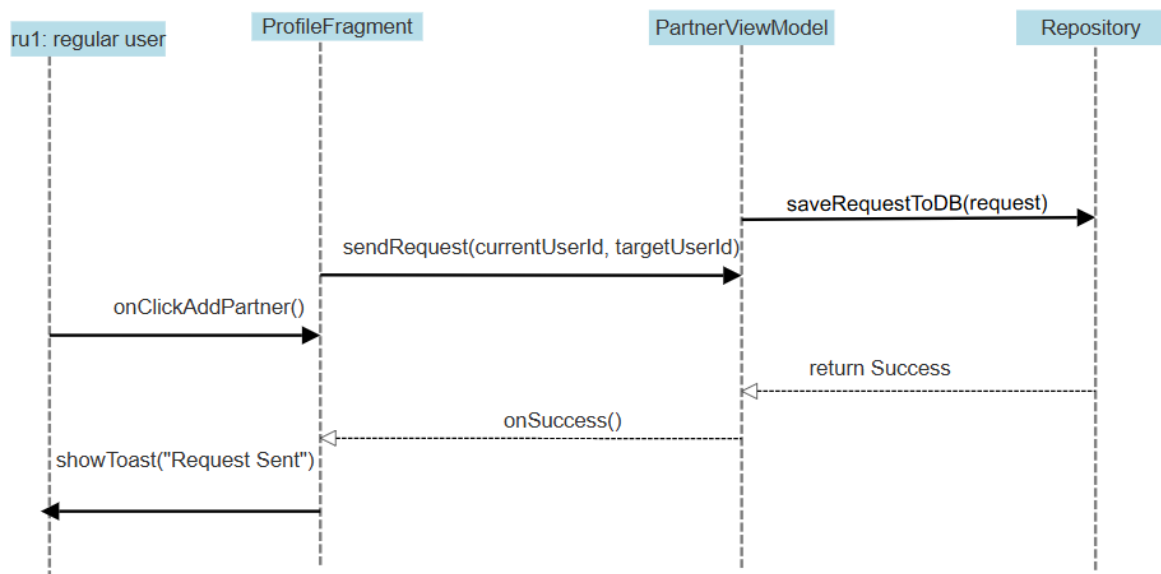
Phase 1 – search for partner



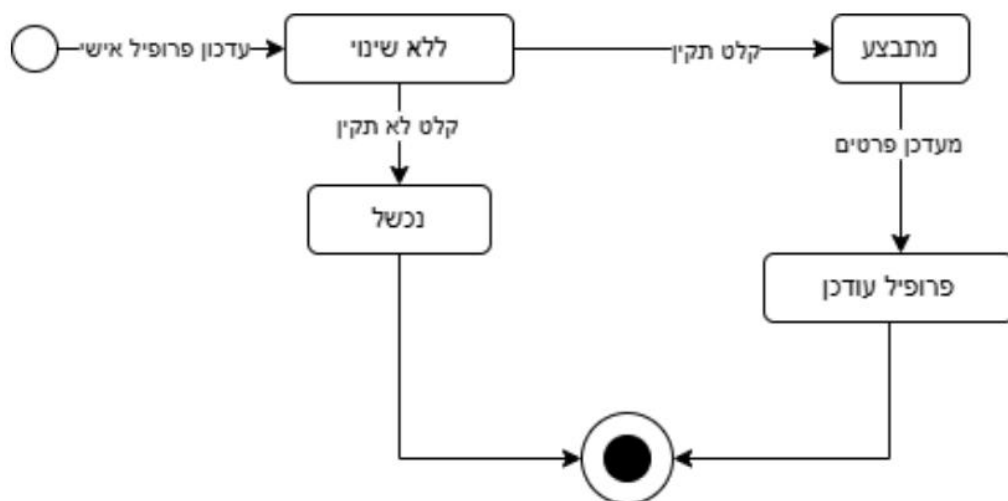
Phase 2 – watch spesific profile



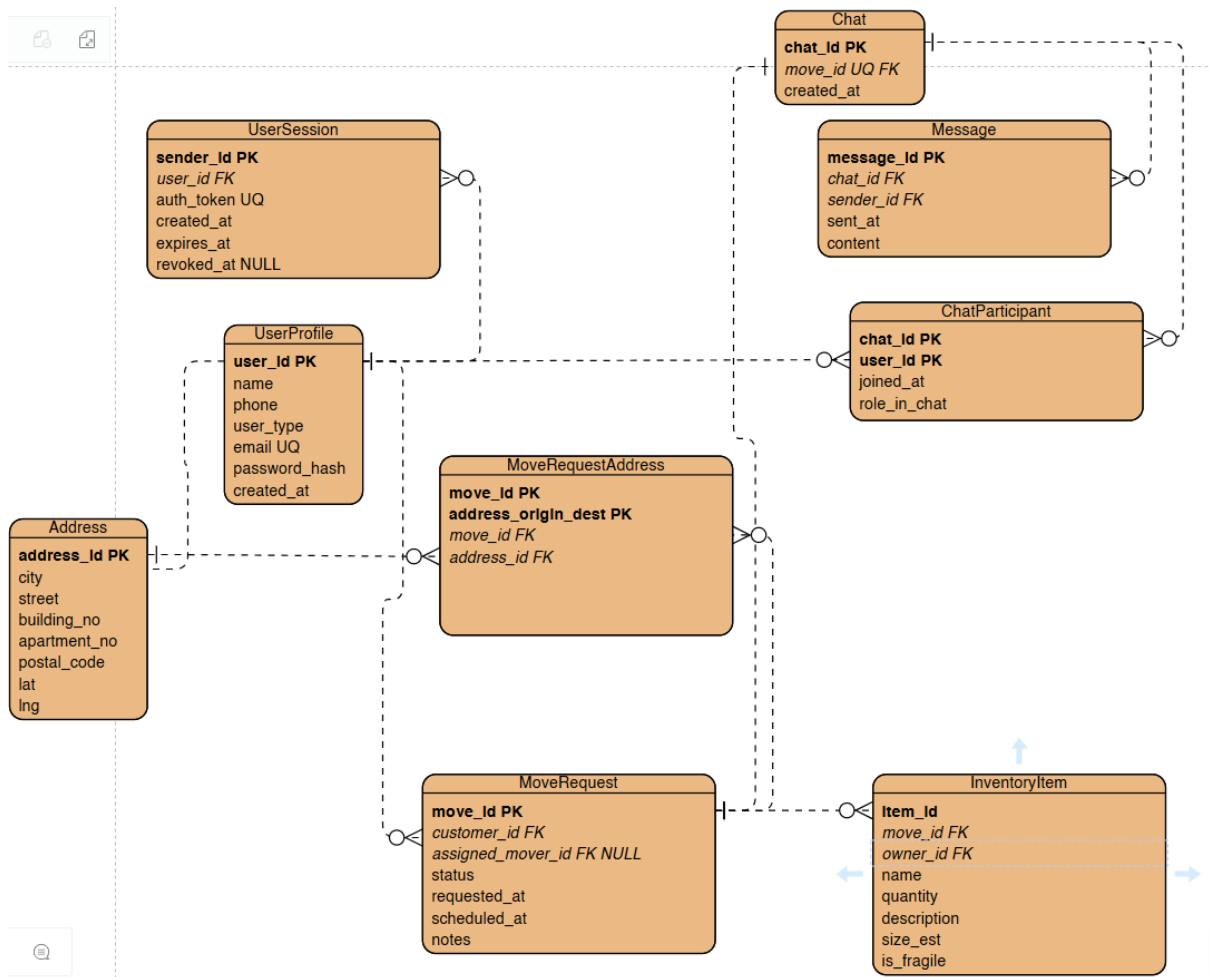
Phase 3 – add partner



5. Machine state diagram – באזור אישי – עדכון פרופיל



6. ERD –



חלק ב'

1. רשימת מסכים:

משותף לשני היוזרים:

מסך הרשמה והתחברות
הרשמה- פרטים אישיים + סימון לקוח\מוביל
התחברות- סיסמא + מייל או דרך גוגל
צ'אטים – רשימת צ'אטים פעילים

תפריט המבורגר צד שמאל למעלה:

אזור אישי - שינוי פרטים אישיים : מייל\טלפון\שם (עבור מוביל יש גם בחירת נק' מוצא של תפעול ורדיוס תפעול)
התנתקות

מסכים לקוח:

סרגל למטה:

בית - המעבר שלי -
כתובת מוצא ויעד (על GPS) עם אפשרות לעריכה (מאפשר לראות פרטי הובלה פעילה או היסטוריה)

חיפוש שותף –

חיפוש הובלה - מכיל שדות Google Autocomplete לבחירת מוצא ויעד, מציג רשימה של מובילים רלוונטיים (לכל מוביל רלוונטי יהיה כרטיס המציג מרחק, דירוג, כפתור "צור קשר" (שפותח צ'אט), כפתור "פרטים" וכפתור "דווח").

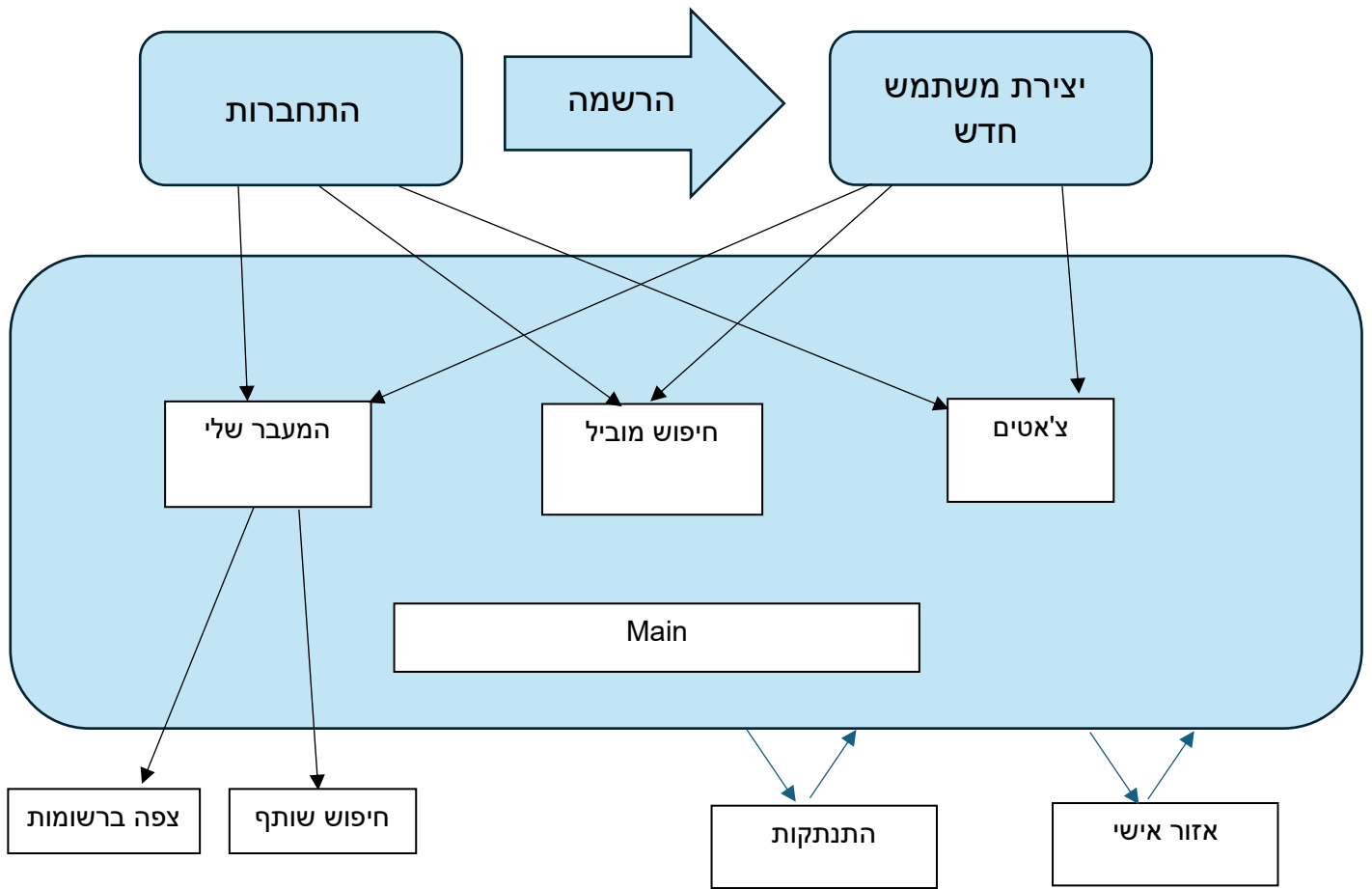
רשומות - יכיל את כל הכרטיסים של הפריטים/קרטונים שהוספנו לאפליקציה.

מסכים מוביל:

סרגל למטה:

בית - ההובלות שלי - מציג רשימה של הובלות פתוחות עם לקוחות. (לכל הובלה פתוחה יהיה כרטיס (רשומה) שיכיל פרטים של ההובלה עבור המוביל וכפתור לפרטים ועריכה שיוביל למסך עם פירוט רחב יותר).

2. תרשים זרימה של מסכים



3. צילומי מסך של המסכים

מסך התחברות/הרשמה:

17:05 82%

התחברות

אימייל

סיסמה

התחבר

- או -

 Sign in with Google

אין לך חשבון? להרשמה לחצי כאן

17:08 83%

הרשמה

אימייל

סיסמה

שם מלא

טלפון

בחרי סוג משתמש:

☒ מוביל ☐ לקוח

כתובת בסיס (חובה למוביל):

לא נבחר מיקום

לחץ לבחירת כתובת במפה

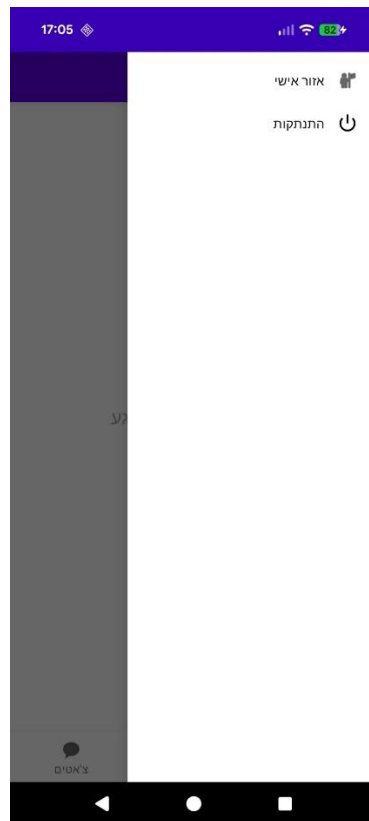
הרשמה

- או -

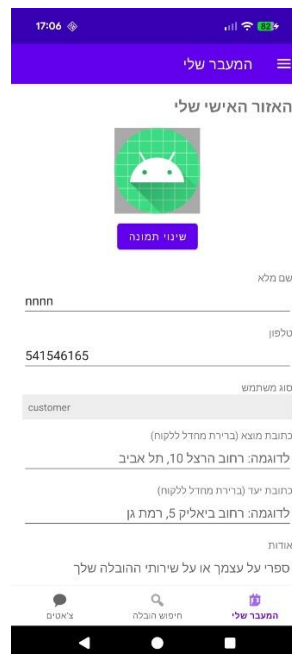
 Sign in with Google

יש לך כבר חשבון? התחברות

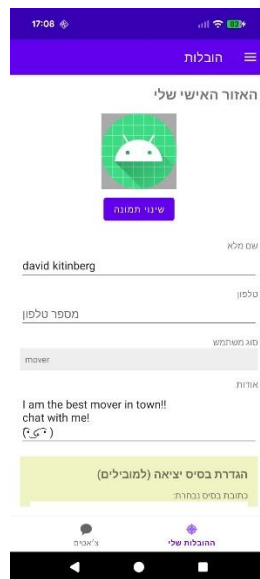
מסך צד של סרגל כלים -



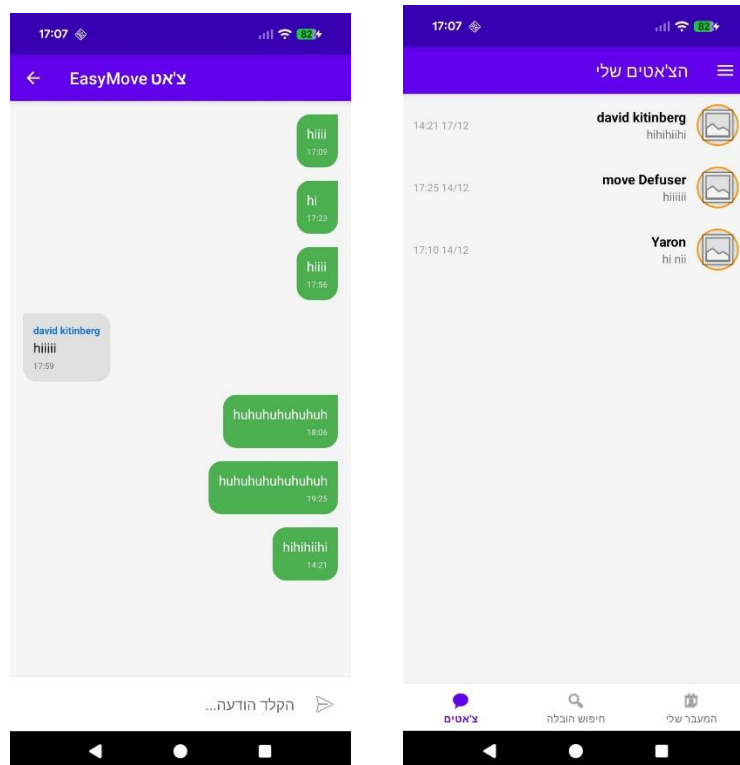
מסך של אזור אישי המוביל/לקוח -



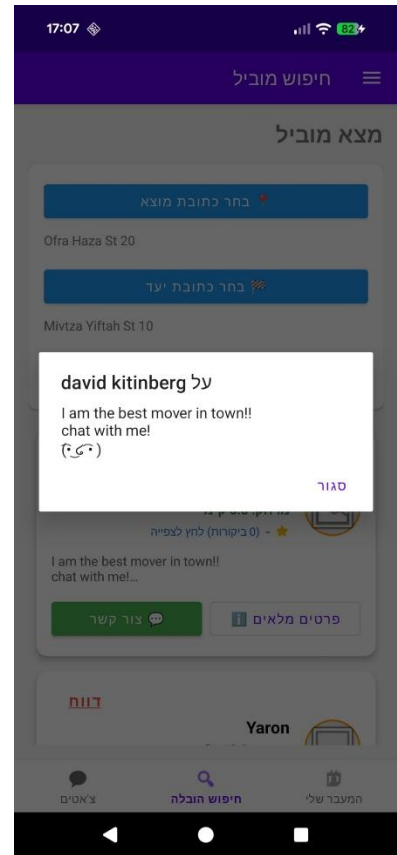
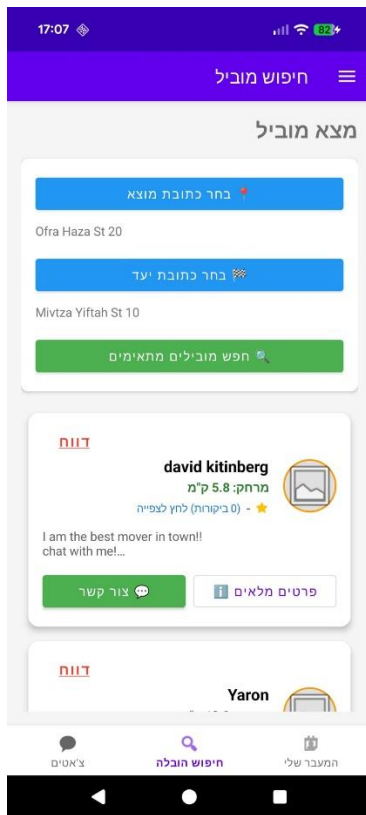
*אזור אישי של הלקוח



מסך של צ'אט ושל "הצ'אטים שלי"



מסכים של חיפוש הובלה –



הסברים על התיקיות ומבנה הקבצים של האפליקציה

בפרויקט EasyMove בחרנו להשתמש בארכיטקטורת MVVM (Model-View-ViewModel), שהיא תבנית עיצוב נפוצה באפליקציות אנדרואיד מודרניות, במיוחד בפרויקטים המבוססים על Jetpack Compose ומשתמשים ב-Firebase.

ה MVVM מפרידה את האפליקציה לשלוש שכבות עיקריות:

Model

שכבה זו כוללת מחלקות נתונים (Data Models) ומחלקות Repository האחראיות על גישה ל-Firebase Authentication ו-Firebase Firestore. שכבה זו כוללת מחלקות נתונים כגון: UserProfile, MoveRequest, InventoryItem, Chat, UserSession ו-Message, בנוסף, קיימת תיקיית repository הכוללת מחלקות כמו UserRepository, MoveRepository, InventoryRepository, ו-ChatRepository, אשר אחראיות על אחזור, יצירה ועדכון נתונים מול Firebase Authentication ו-Firebase Firestore.

View

שכבה זו כוללת את כל קבצי הממשק המשתמש, כגון מסכי Jetpack Compose וקומפוננטות גרפיות כלליות (לדוגמה: סרגל ניווט תחתון). הקבצים מחולקים ל-Activities ו-Fragments. בשכבה זו נמצאים מסכים המציגים רשימות פריטים, מסך ניהול "המעבר שלי", מסכי חיפוש מובילים, פרופילים של מובילים ומסך צ'אט. תפקידה של שכבת ה View הוא להציג מידע למשתמש ולהגיב לפעולות כמו לחיצות וניווט.

ViewModel

שכבת ה ViewModel אחראית על ניהול ה state של המסכים, טיפול בלוגיקת האפליקציה ותיווך בין שכבת ה View לשכבת ה Model. לכל מסך מרכזי באפליקציה קיים ViewModel ייעודי, אשר שומר את המידע הרלוונטי בזיכרון, פונה לשכבת ה Repository לצורך שליפת נתונים, ומעדכן את ה UI בצורה ריאקטיבית בהתאם לשינויים בנתונים. בעצם ה ViewModel לא מצייר דברים למסך, הוא פשוט מחזיק מידע, שומר עליו, ומעדכן את הקומפוננטות בכל פעם שהמידע משתנה.

בחרנו בארכיטקטורת MVVM מאחר שהיא מתאימה במיוחד לאפליקציות מבוססות Jetpack Compose, ומאפשרת הפרדה ברורה בין שכבות המערכת, דבר שתואם באופן ישיר לדרישות של הפרויקט שלנו EasyMove.

המערכת שלנו כוללת ממשק משתמש אינטראקטיבי ודינמי (UI), גישה לנתונים באמצעות Firebase Authentication ו-Firebase Firestore, וניהול של מצבים (state) מרובים כגון: הצגת רשימות פריטים/קרטונים לניהול מעבר דירה, ניהול אזור "המעבר שלי", חיפוש/הצגת מובילים לפי מיקום, פרופילים של מובילים (דירוג/חוות דעת/אודות), פתיחת בקשות הובלה, וכן צ'אט פנימי בין לקוח למוביל.

ה MVVM מאפשרת לנו:

- לשמור את הקוד מודולרי ומופרד - שכבת ה UI מציגה מידע בלבד ואינה מטפלת בשליפת נתונים או בלוגיקה עסקית. כל גישה ל-Firebase מתבצעת דרך שכבות הנתונים (Model/Repository).

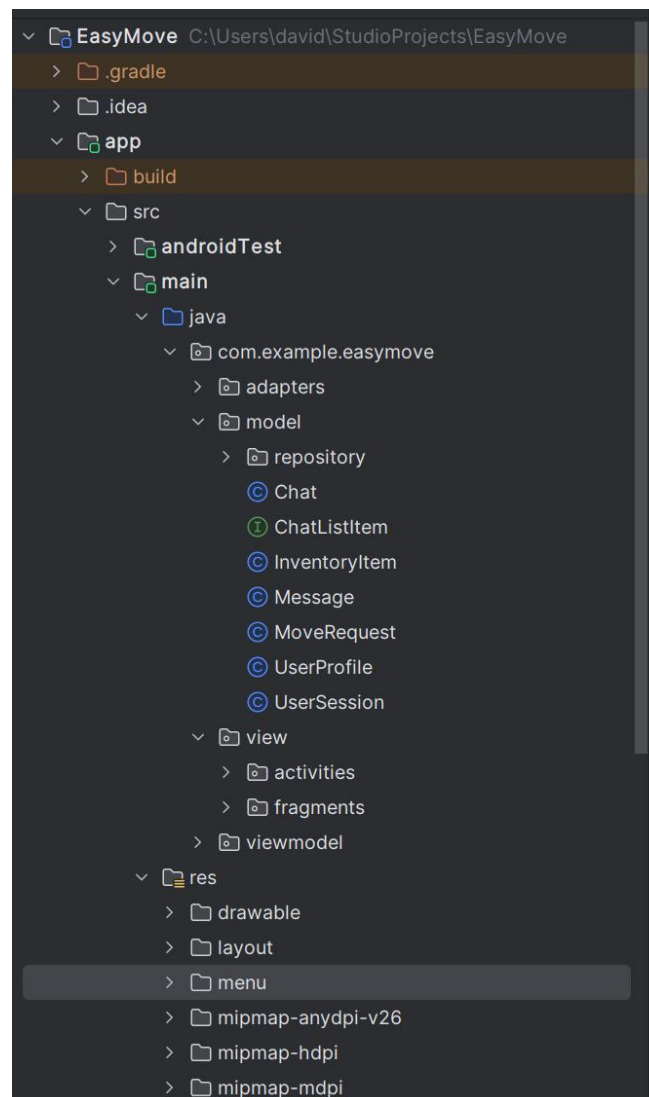
- להתמודד עם ריבוי מצבים בצורה מסודרת - לכל מסך/פיצ'ר יש ViewModel משלו שמנהל state (למשל: טעינה, הצלחה, שגיאה, תוצאות חיפוש, רשימות פריטים, הודעות צ'אט), וכך קל יותר לשלוט בהתנהגות המסכים.

- לשפר את היכולת לבדוק ולתחזק את הקוד- הלוגיקה מרוכזת בViewModel ולא בתוך מסכי הUI, ולכן ניתן לבדוק אותה בצורה נקייה יותר ולבצע שינויים בלי "לשבור" את הממשק.

- להתאים לקונספט של Compose שבו המידע זורם למסך ומתעדכן ריאקטיבית. כלומר, כאשר ה-ViewModel מעדכן את הstate (למשל אחרי שליפה מ-Firebase), מסכי Compose מתעדכנים אוטומטית בלי צורך לנהל עדכונים ידניים.

כך אנחנו עומדים בדרישות של UI מודרני, ניהול State חכם, וניהול מידע מבוסס Firebase, תוך שמירה על קוד קריא, נוח להרחבה, ומתאים לעבודה בצוות לאורך זמן.

צילום מסך של התיקיות:



היתרונות המרכזיים בארכיטקטורת MVVM

הארכיטקטורה שבחרנו לפרויקט היא MVVM משום שהיא מספקת יתרונות משמעותיים במספר היבטים מרכזיים:

תחזוקה (Maintainability)

הפרדת האחריות בין View, ViewModel ו-Model מאפשרת שינוי ועדכון של לוגיקה או מקורות מידע מבלי להשפיע על שכבת ה-UI. בכך נשמר קוד קריא, ברור וקל לתחזוקה לאורך זמן.

הרחבה (Scalability)

בעת הוספת תכונות חדשות, כגון מערכת דירוגים, התראות, צ'אט פנימי או סינון מתקדם, ניתן להוסיף ViewModels ומסכים חדשים מבלי לשנות קוד קיים. כל מסך מתפקד כמודול עצמאי עם לוגיקה נפרדת.

ביצועים (Performance)

ה-ViewModel שומר את ה-state בזיכרון גם בעת מעבר בין מסכים, ולכן נמנעת טעינה חוזרת ומיותרת של נתונים מ-Firebase.

בנוסף, השילוב בין MVVM לבין מנגנון ה-State של Jetpack Compose גורם לעדכון ה-UI רק כאשר הנתונים משתנים, מה שמשפר את תגובתיות האפליקציה ומפחית עומס מיותר.